

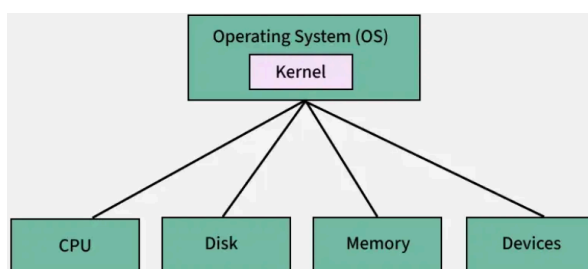
Arquitectura del Kernel, Dispositivos y Optimización de E/S

El Kernel [capítulo 2]

El kernel es el núcleo de un sistema operativo (SO) y actúa como intermediario entre el hardware y las aplicaciones de usuario. Su principal responsabilidad es garantizar que los recursos críticos del sistema —CPU, memoria, dispositivos de entrada/salida y procesos— funcionen de manera eficiente y segura. El kernel administra la ejecución de múltiples procesos simultáneamente, controla el acceso a memoria y dispositivos, y proporciona mecanismos de comunicación y sincronización entre procesos.

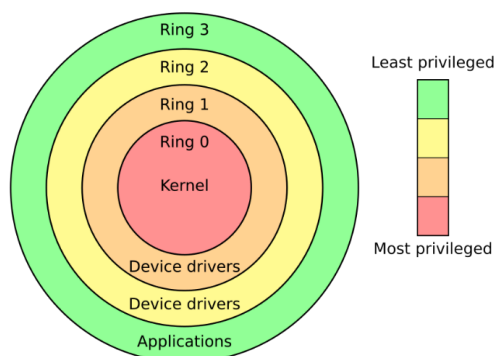
Gestión	Función	Ejemplo
Procesos	Crear y administrar procesos	fork(), kill
Memoria	Asignar y liberar memoria	malloc, free
E/S	Administrar dispositivos	USB, teclado, pantalla
Sistema de archivos	Organizar y proteger archivos	ls, chmod
Seguridad	Controlar acceso y autenticación	ACL, passwd

El kernel es el corazón del sistema operativo, permitiendo que múltiples procesos compartan recursos sin interferencias y asegurando la estabilidad del sistema.



Arquitectura de Privilegios en Procesadores x86

El objetivo principal de los niveles de privilegio en los procesadores modernos es garantizar *seguridad y aislamiento* entre el sistema operativo y las aplicaciones. Gracias a este modelo, las operaciones críticas (como modificar registros del procesador, acceder directamente a la memoria o interactuar con dispositivos de hardware) solo pueden ejecutarse desde los niveles de mayor privilegio. Así, si una aplicación de usuario —por ejemplo, un navegador web— falla o es comprometida, no debería ser capaz de afectar directamente al núcleo del sistema operativo (kernel) ni a otros procesos en ejecución.



Esquema de Anillos de Protección: Los procesadores de la familia Intel x86 y x86_64 fueron diseñados con un esquema de cuatro anillos de protección (del 0 al 3), que representan distintos niveles de acceso al hardware

Aunque la arquitectura x86 ofrece cuatro anillos, la mayoría de los sistemas operativos contemporáneos, incluidos Linux y Windows, utilizan únicamente dos niveles:

- **Ring 0 (Modo Kernel):** Control total del hardware y de las instrucciones privilegiadas de la CPU. El kernel y sus controladores residen en este nivel
- **Ring 3 (Modo Usuario):** Nivel menos privilegiado. Acceso restringido al hardware. Se ejecutan aplicaciones (Firefox, Chrome, LibreOffice, juegos, Python, Java).

Los anillos intermedios (Ring 1 y Ring 2) fueron diseñados para controladores o servicios de sistema con privilegios parciales, pero rara vez se usan. La simplificación a solo dos niveles reduce complejidad y mejora la seguridad.

De esta forma, la arquitectura de privilegios en procesadores x86 logra un equilibrio entre rendimiento, simplicidad y protección, constituyéndose en la base del modelo de seguridad adoptado en los sistemas operativos modernos.

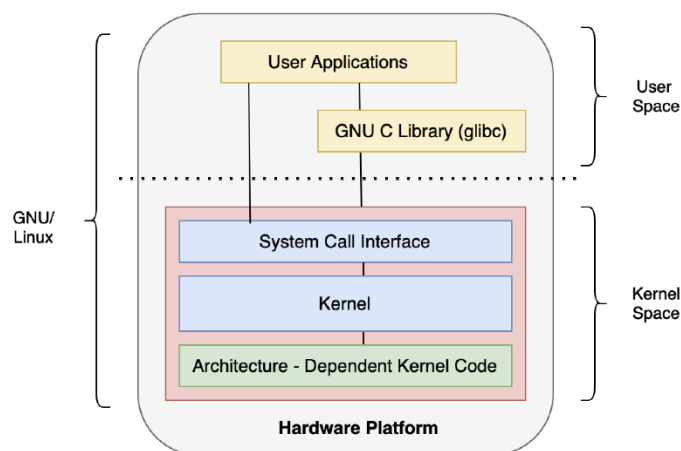
Un aspecto relevante de x86 es que el procesador identifica el modo actual a través del *registro de segmento de código* (CS). El campo de privilegio dentro de este registro determina si la CPU está ejecutando en Ring 0 o Ring 3.

Ejemplo con IRET en x86: Cuando ocurre una transición de usuario a kernel (por ejemplo, al ejecutar una system call), la CPU realiza los siguientes pasos:

1. Cambia el valor del CS al selector correspondiente a Ring 0.
2. Guarda el contexto del proceso (puntero de instrucción EIP/RIP, flags, pila de usuario).
3. Ejecuta el handler en el kernel.
4. Una vez que la operación concluye, la instrucción IRET (Interrupt Return) restaura el valor previo del CS y del puntero de instrucción, devolviendo el control al programa en Ring 3.

Este mecanismo asegura que una aplicación nunca pueda volver por sí sola a Ring 0: solo el kernel puede ejecutar IRET y validar la transición de regreso a Ring 3

Modos de ejecución Usuario - Kernel



Los sistemas operativos modernos utilizan *dos modos de ejecución* definidos por la arquitectura del procesador:

- **Modo Kernel:**

Es el entorno de máxima confianza del procesador. El código que corre en este modo puede:

- ejecutar cualquier instrucción de la CPU (incluyendo las instrucciones privilegiadas),
- acceder directamente a cualquier dirección de memoria física,
- interactuar con el hardware (discos, interfaces de red, timers, controladores de E/S).

En este modo corre el núcleo del sistema operativo (kernel): gestión de procesos, administración de memoria, control de dispositivos y seguridad. También corren aquí los módulos del kernel (por ejemplo, drivers de hardware).

El código en modo kernel tiene acceso total. Un error aquí puede comprometer todo el sistema.

- **Modo Usuario:**

Es el entorno restringido donde corren las aplicaciones: navegadores, editores, intérpretes de Python, herramientas de línea de comandos (ls, grep), máquinas virtuales Java (JVM), etc.

En este modo, el procesador bloquea las instrucciones privilegiadas. Si una aplicación intenta ejecutar una instrucción que requiere modo kernel, el procesador genera una excepción y el sistema operativo la termina o la penaliza.

Para acceder a recursos controlados (archivos, red, memoria compartida), las aplicaciones no tienen otra opción que pedirselo al kernel mediante un mecanismo controlado: las llamadas al sistema (system calls).

Componentes Clave entre Usuario–Kernel

Aplicaciones de Usuario

Son los programas que el usuario ejecuta directamente en el *espacio de usuario*. Como por ejemplo Chrome, Firefox, LibreOffice, juegos, comandos como [ls](#), [grep](#), [python](#).

Un programa en **C** invoca funciones de bibliotecas como [glibc](#). **Python** o **Java** se ejecutan en intérpretes o máquinas virtuales (CPython, JVM), también en user space.

Por ejemplo: Un script en Python que abre un archivo llama internamente a [open\(\)](#). No accede al hardware, sino que pasa por bibliotecas, luego system call y finalmente al kernel.

Bibliotecas de Usuario (User Libraries)

En la práctica, las aplicaciones no invocan directamente al kernel, sino que utilizan *bibliotecas de usuario* como [glibc](#) en Linux. Estas librerías cumplen tres funciones:

1. Abstracción: el programador usa funciones de alto nivel ([fopen](#), [printf](#), [os.read](#)) sin preocuparse por los detalles internos. Por ejemplo

```
int fd = open("datos.txt", O_RDONLY);
```

Aquí no se llama directamente al kernel, sino a la función `open()` de **glibc**.

2. Traducción en la librería: la librería **glibc** empaqueta los parámetros y traduce esas funciones a system calls concretas ([sys_open](#), [sys_write](#)).

3. Transición a modo kernel: La librería ejecuta una instrucción especial de la CPU (ej. `int 0x80` en x86 o `syscall` en x86_64) que transfiere el control al kernel.

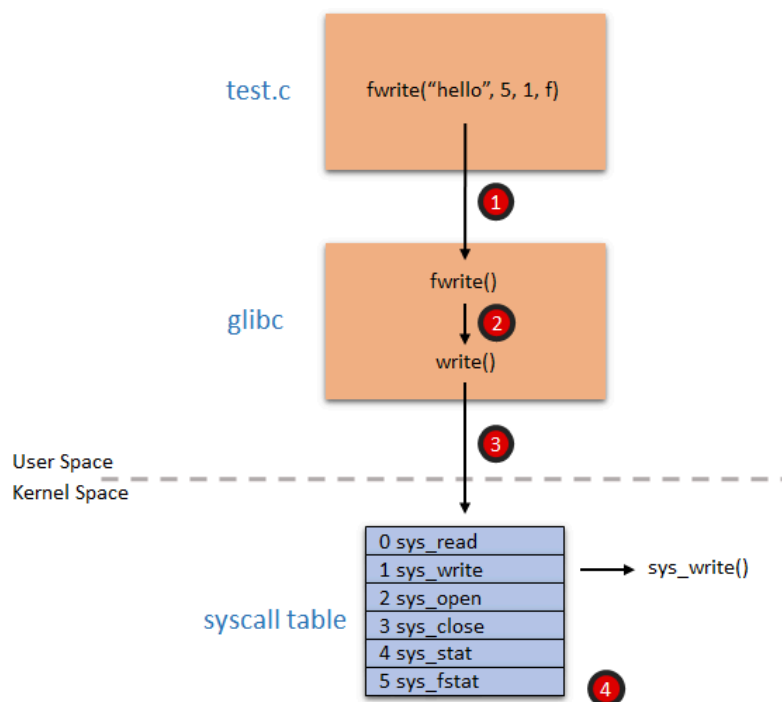
4. Resolución dentro del kernel: El número de la system call (guardado en un registro, como `eax` en x86) indica al kernel cuál rutina debe ejecutarse.

- El kernel mantiene internamente una tabla de punteros a sus system calls, inaccesible desde el espacio de usuario.
- El handler adecuado se ejecuta, validando permisos y realizando la operación solicitada.

5. Retorno al user space: Una vez completada la operación, el kernel devuelve el resultado a la librería, que lo entrega a la aplicación en **modo usuario**.

Frontera entre Usuario y Kernel

Las aplicaciones en modo usuario necesitan acceder a recursos controlados (archivos, sockets, memoria compartida). Sin embargo, como no pueden ejecutar instrucciones privilegiadas directamente, deben hacerlo mediante *llamadas al sistema* (*system calls*).



<https://www.sysdig.com/blog/fascinating-world-linux-system-calls>

En Linux (arquitectura x86/x86_64), las system calls se organizan en una estructura llamada:

```
extern void *sys_call_table[];
```

La tabla real dentro del kernel

Esta tabla contiene punteros a funciones internas del kernel. Cada índice corresponde al número de system call, y apunta a la rutina que se implementa.

Función de usuario	handlers de system call	Descripción
open()	sys_open()	Abrir un archivo y obtener un descriptor.
read()	sys_read()	Leer datos de un descriptor de archivo.
write()	sys_write()	Escribir datos en un descriptor.
close()	sys_close()	Cerrar un descriptor de archivo.
lseek()	sys_lseek()	Mover el puntero de lectura/escritura en el archivo.
stat()	sys_stat()	Consultar metadatos del archivo (permisos, tamaño, fecha).

Transiciones de Modo Usuario a Modo Kernel

El procesador cambia a modo kernel únicamente en escenarios controlados por el hardware y el sistema operativo. Esto garantiza que las aplicaciones de usuario no puedan ejecutar código privilegiado de manera arbitraria, preservando la seguridad y estabilidad del sistema. Existen tres causas para cambiar de **modo usuario** a **modo kernel**:

1. System calls:
2. Excepciones síncronas:
3. Interrupciones externas

1. System Calls

Cuando un programa en modo usuario necesita un servicio del kernel (por ejemplo, abrir un archivo, leer datos, crear un proceso), utiliza una system call. Ejemplo en x86: int 0x80 (32-bit), syscall (x86_64).

Flujo:

1. El proceso invoca la system call en user space.
2. La CPU realiza un salto controlado a código privilegiado en Ring 0 (modo kernel).
3. El kernel valida parámetros y permisos, ejecuta el handler correspondiente (por ejemplo, sys_open() para abrir un archivo).
4. Una vez completada la operación, la CPU utiliza la instrucción IRET para restaurar el contexto y regresar al modo usuario.

El usuario nunca elige directamente a qué código privilegiado saltar; esto está definido por la Interrupt Descriptor Table (**IDT**) del kernel.

2. Excepciones Síncronas

Una excepción síncrona ocurre cuando el código en ejecución provoca una condición que debe ser atendida inmediatamente por el kernel. Estas son generadas por el CPU en respuesta a errores o eventos inesperados dentro del flujo del programa. Por ejemplo: *Page Fault*, el programa intenta acceder a memoria no mapeada. *Divide by Zero*, intento de división entre cero.

Flujo de atención:

1. La CPU detecta la excepción y guarda el contexto del proceso (registros CS, RIP/EIP, EFLAGS).
2. Se transfiere el control al handler del kernel definido
3. El kernel atiende la excepción, ya sea asignando memoria, enviando señales o terminando el proceso.
4. Si es seguro continuar, la CPU retorna a modo usuario

3. Interrupciones Asíncronas

Una interrupción asíncrona es generada por un dispositivo hardware externo, independiente del flujo de ejecución del programa. Por ejemplo: Temporizadores del sistema (timer interrupts), Paquetes de red entrantes (NIC interrupts) y Actividad de teclado o mouse.

Flujo de atención:

1. La CPU suspende temporalmente el proceso en modo usuario.
2. Cambia al Ring 0 (modo kernel).
3. Ejecuta el interrupt handler correspondiente del kernel.
4. Una vez finalizada la atención, el CPU retorna al proceso usuario

La transición está determinada por la arquitectura y la IDT; el usuario no controla ni puede saltar arbitrariamente al código privilegiado.

System Call

Una llamada al sistema es un mecanismo mediante el cual un programa de usuario solicita al núcleo del sistema operativo (kernel) que realice una operación específica, como acceder a archivos, gestionar procesos o controlar dispositivos. Estas llamadas permiten a los programas interactuar con el sistema operativo y obtener servicios esenciales para su ejecución. El proceso inicia cuando el programa ejecuta una instrucción que desencadena un cambio al modo kernel. El sistema operativo valida la solicitud, realiza la operación y devuelve el resultado al programa.

Descriptor de archivo

Un descriptor de archivo (*file descriptor* o *fd*) es un número entero no negativo que el kernel asigna cuando un proceso abre un archivo. Actúa como un índice en la tabla de descriptors de archivos del proceso, donde el kernel mantiene la asociación entre cada descriptor y el archivo o recurso correspondiente. Los descriptors permiten que el proceso realice operaciones como (read(), write(), close(), etc.) se realicen de forma controlada por el kernel.

- Cada proceso tiene su propia tabla, creada por el kernel al iniciar el proceso.
- Cada entrada en la tabla apunta a un objeto abierto en el kernel: archivo, socket, pipe, etc.

Asignación y tabla de descriptors

El kernel asigna el descriptor de archivo a nivel de proceso, no de archivo. Esto significa que cada proceso tiene su propia tabla de descriptors, y el descriptor solo es válido para el proceso que llamó a open().

El valor de los descriptors comienza desde 0, 1 y 2, que están reservados para:

- **0** – Entrada estándar (stdin)

- 1 – Salida estándar (stdout)
- 2 – Error estándar (stderr)

Los descriptors mayores a 2 (3, 4, 5, ...) no tienen nombres fijos. Su significado depende de lo que el proceso haya abierto y en qué orden.

Cada proceso tiene su propia tabla de descriptors, por lo que fd=3 en un proceso no necesariamente corresponde al mismo recurso en otro.

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    // Abrir dos archivos
    int fd1 = open("pagina.html", O_RDONLY); // Simula navegador abriendo un
    archivo cacheado
    int fd2 = open("notas.txt", O_WRONLY | O_CREAT, 0644); // Simula editor
    guardando notas

    if (fd1 < 0 || fd2 < 0) {
        perror("open");
        return 1;
    }

    printf("Descriptor de pagina.html: %d\n", fd1);
    printf("Descriptor de notas.txt: %d\n", fd2);

    char buffer[128];
    int bytes = read(fd1, buffer, sizeof(buffer)); // Leer de pagina.html
    write(fd2, buffer, bytes); // Escribir lo leído en notas.txt

    close(fd1);
    close(fd2);

    return 0;
}
```

Salida:

- *Descriptor de pagina.html: 3*
- *Descriptor de notas.txt: 4*

[pagina.html](#) (un archivo simulado de caché de navegador) devuelve descriptor 3.

[notas.txt](#) (un archivo de un editor de texto) devuelve descriptor 4.

dup() y dup2()

Cuando un proceso abre un archivo, el kernel crea un **objeto de archivo interno** que contiene información como:

1. **Offset de lectura/escritura:** posición actual dentro del archivo donde se leerá o escribirá el siguiente byte. Por ejemplo, si abrimos `archivo.txt` y hacemos un `read(fd, buf, 10)`, el offset avanza 10 posiciones.
2. **Permisos de acceso:** determina si el descriptor puede leer, escribir o ambos. Por ejemplo, `O_RDONLY` para solo lectura, `O_WRONLY` para solo escritura, o `O_RDWR` para ambos.
3. **Contador de referencias:** indica cuántos descriptors (posiblemente de distintos procesos) apuntan al mismo objeto del kernel.

```
int fd = open("archivo.txt", O_RDWR | O_CREAT, 0644);
```

- Se asigna un descriptor (por ejemplo 3).
- Offset inicial = 0.
- Permisos = lectura y escritura.

Ahora, si hacemos:

```
write(fd, "Hola", 4);
```

- Se escriben 4 bytes y el offset avanza a 4.
- Si hacemos otra escritura, comenzará en la posición 4

dup(fd)

Cuando usamos `dup(fd)`, el sistema hace lo siguiente:

1. Busca el primer descriptor libre del proceso (mayor o igual a 0).
2. Crea un nuevo descriptor que apunta al mismo objeto del kernel que `fd`.
3. El offset y los permisos se comparten: si lees 10 bytes desde el nuevo descriptor, el offset del objeto del kernel se actualiza y afecta a ambos descriptors

```
int fd1 = open("archivo.txt", O_RDWR | O_CREAT, 0644); // fd1 = 3
int fd2 = dup(fd1); // fd2 = 4
```

- `fd1` y `fd2` apuntan al mismo archivo en el kernel.
- Si escribimos con `fd1`:

```
write(fd1, "ABCD", 4); // offset = 4
```

Luego leemos con `fd2`:

```
char buf[5];
lseek(fd2, 0, SEEK_SET); // Opcional: reinicia offset
read(fd2, buf, 4); // leemos "ABCD"
```

Observa que el offset es compartido. Si no hacemos `lseek`, `fd2` comenzaría en `offset = 4`, no al inicio.

dup2(fd, newfd)

dup2(fd, newfd) copia fd sobre un descriptor específico newfd:

1. Si newfd estaba en uso, primero se cierra automáticamente.
2. Luego newfd apunta al mismo objeto que fd.
3. Offset y permisos también se comparten.

Ejemplo clásico de redirección en shell:

```
int fd = open("salida.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fd, STDOUT_FILENO); // ahora stdout (1) apunta a salida.txt
close(fd);
printf("Hola mundo\n"); // se escribe en salida.txt
```

- Antes de dup2, stdout = descriptor 1 apuntando a pantalla.
- Después de dup2, stdout = descriptor 1 apuntando a salida.txt.
- fd original (3 en este caso) y descriptor 1 ahora comparten offset y permisos.

Funciones Básicas de E/S en UNIX [\[Capítulo 3\]](#)

Las operaciones de entrada y salida (E/S) en sistemas UNIX se realizan mediante un conjunto de llamadas al sistema (system calls) que permiten interactuar directamente con archivos, dispositivos y otros recursos. Estas llamadas permiten realizar operaciones de lectura, escritura, manipulación de la posición de lectura/escritura y cierre de descriptores.

A continuación presentamos las categorías de System Calls mas comunes:

- *Gestión de archivos*: open(), read(), write(), close(), lseek()
- *Gestión de procesos*: fork(), exec(), wait()
- *Gestión de dispositivos*: ioctl()
- *Redes*: socket(), bind(), listen(), accept()
- *Memoria*: mmap(), brk()

1. open()

La función open() abre un archivo o dispositivo y devuelve un descriptor de archivo. Si el archivo no existe y se usa la bandera O_CREAT, el sistema crea el archivo.

```
int fd = open("archivo.txt", O_CREAT | O_WRONLY, 0644);
```

- fd → Descriptor de archivo.

Parámetros

- O_RDONLY → Abre el archivo en modo solo lectura
- O_WRONLY → Abre el archivo en modo solo escritura
- O_RDWR → Abre el archivo en modo lectura/escritura
- O_CREAT → Crea el archivo si no existe.
- 0644 → Permisos (rw-r--r--)

Retorno:

- Retorna el descriptor de archivo (entero positivo) si tiene éxito.

- Retorna -1 si ocurre un error (se guarda el valor en errno)

2. read()

Lee datos de un archivo en un búfer. El kernel ajusta el desplazamiento (offset) y devuelve el número de bytes leídos o 0 si está en el final del archivo (EOF).

```
char buffer[100];  
int n = read(fd, buffer, 100);
```

Parámetros:

- fd → Descriptor de archivo.
- Buffer → Buffer donde se almacenan los datos leídos
- 100 → Número máximo de bytes a leer

Retorno:

- Número de bytes leídos si tiene éxito.
- 0 si alcanza el EOF.
- -1 si ocurre un error.

3. write()

Escribe datos desde un búfer hacia el archivo. Si hay espacio, escribe directamente. Si hay errores o el archivo está lleno, devuelve -1.

```
write(fd, "Hola mundo", 10);
```

Parámetros:

- fd → Descriptor de archivo.
- "Hola mundo" → Buffer de datos a escribir.
- 10 → Número de bytes a escribir.

Retorno:

- Número de bytes escritos si tiene éxito.
- -1 si hay error (como archivo de solo lectura).

4. lseek()

Mueve el puntero de lectura/escritura a una posición específica dentro del archivo, siendo particularmente útil en dispositivos de bloques para acceder directamente a cualquier bloque de datos sin necesidad de leer o escribir todos los bloques anteriores. Permite posicionar el puntero más allá del final, creando "huecos" (holes).

```
lseek(fd, 0, SEEK_END);
```

Parámetros:

- fd → Descriptor de archivo.
- 0 → Desplazamiento.
- SEEK_SET → Desde el inicio del archivo.
- SEEK_CUR → Desde la posición actual.
- SEEK_END → Desde el final del archivo.

5. close()

Cierra el descriptor y libera los recursos asociados. Si hay cambios pendientes, el kernel los sincroniza antes de cerrar.

```
close(fd);
```

Retorno:

- 0 si tiene éxito.
- -1 si hay error.
- Descriptor de archivo.

Funciones Adicionales

6. ioctl()

La llamada ioctl() es especialmente útil para realizar operaciones específicas de control que no están cubiertas por las funciones estándar de lectura y escritura. Estas operaciones deben hacerse públicas y documentadas para evitar conflictos entre diferentes drivers

```
ioctl(fd, FIONBIO, &flags);
```

Parámetros:

- Descriptor de archivo.
- Comando específico.
- Parámetro adicional.

7. dup()

Duplica el descriptor de archivo hacia el valor más bajo disponible. Ambos descriptors comparten el mismo desplazamiento y flags.

7. dup2()

Funciona como dup() pero permite especificar el valor del descriptor de destino. Si el descriptor de destino está abierto, lo cierra primero.

8. fcntl()

Permite modificar atributos de un descriptor, como modos de acceso y flags, o establecer bloqueos (locks).

Ciclo de Vida de un Descriptor

1. **Apertura (open):** El kernel crea una nueva entrada en la tabla de archivos abiertos y asocia el descriptor.
2. **Operaciones (read, write, lseek):** Modifican el desplazamiento y afectan la estructura de la tabla.
3. **Cierre (close):** El kernel decrementa el contador de referencias. Si el contador llega a cero, libera el descriptor.

Nota: no todos los dispositivos siguen el ciclo completo de cinco pasos (open(), read(), write(), lseek(), close()). El ciclo depende del tipo de dispositivo y de cómo se gestiona en el kernel:

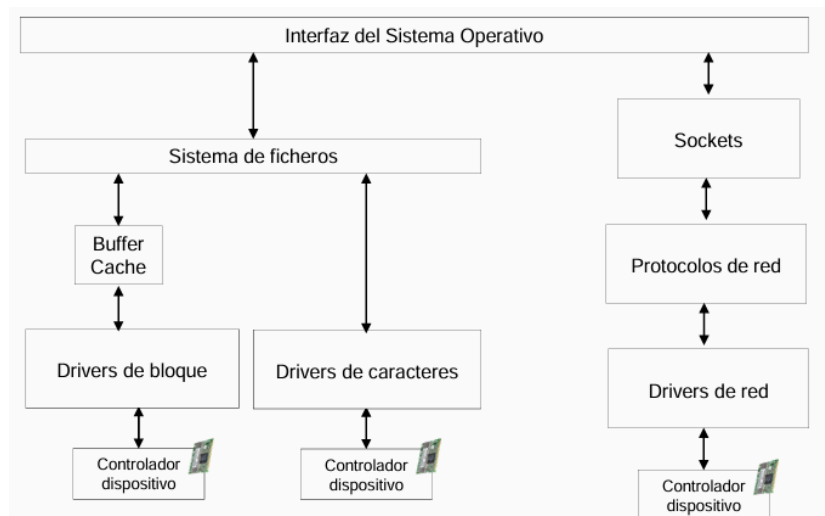
- *Teclado* → Solo admite read() porque no es un dispositivo que acepte escritura o búsqueda (lseek() no tiene sentido).
- *Sockets* → Admiten read(), write() y close() pero no lseek() porque no tienen un "desplazamiento" en el flujo de datos.

- Tuberías (pipes) → Solo permiten read() y write() (no lseek() ni open() directo, ya que se crean en tiempo de ejecución).
- *Dispositivos de bloque* → Como discos duros, permiten read(), write(), lseek(), y otras operaciones como ioctl() para configuración.
- Dispositivos de carácter → Como puertos serie o teclados, permiten read() y write() pero lseek() puede no ser válido.

El kernel define el comportamiento de cada dispositivo según la interfaz del controlador (driver) correspondiente

Gestión de Controladores (Drivers)

Los drivers son componentes esenciales en los sistemas operativos, especialmente en Linux, donde están profundamente integrados con el núcleo (kernel) del sistema. Los drivers permiten la comunicación entre el hardware y el software, gestionando cómo los dispositivos externos, como discos duros, teclados y tarjetas gráficas, interactúan con el sistema



Modelos de Drivers

Los drivers en Linux pueden estar en dos espacios diferentes. *Drivers en espacio de usuario:* Estos interactúan con el hardware a través del kernel, pero no requieren ejecución directa en el núcleo. Son más fáciles de modificar y menos propensos a causar fallos graves. *Drivers en espacio de kernel:* Se ejecutan directamente en el núcleo del sistema, lo que les permite un control más completo sobre el hardware, aunque también pueden causar fallos graves si no están bien gestionados.

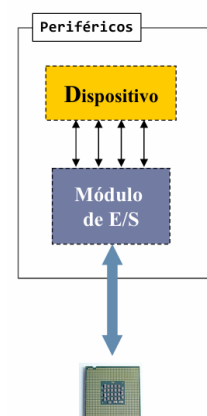
Estructura general de un periférico

Los *periféricos* son dispositivos externos conectados a una computadora que permiten almacenar información o comunicarse con el mundo exterior.

Un periférico está compuesto por dos partes: **Periférico = Dispositivo + Controlador**

Periférico: En este caso, el periférico completo es el disco duro. Es un dispositivo externo que se conecta a la computadora y sirve para almacenar información de manera permanente.

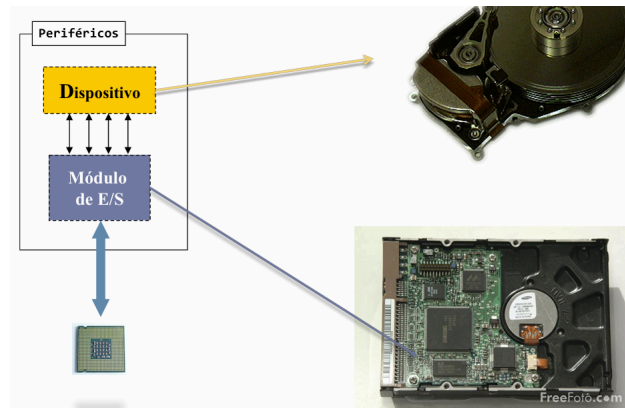
Dispositivo: La parte amarilla representa el dispositivo en sí, es decir, los componentes mecánicos y electrónicos que permiten el almacenamiento y la



lectura de datos. En el caso del disco duro, esto incluye los platos magnéticos, el cabezal de lectura/escritura y el motor.

Módulo de E/S: Representado por el rectángulo azul, es la interfaz entre el dispositivo y la CPU (Unidad Central de Procesamiento). Este módulo, también llamado controlador, se encarga de traducir las señales entre la CPU y el dispositivo, permitiendo que ambos se comuniquen de manera efectiva.

CPU : La Unidad Central de Procesamiento (CPU) solicita datos al dispositivo o envía comandos de escritura. El módulo de E/S (controlador) traduce estas solicitudes en señales que el disco duro entiende y luego retorna los datos solicitados a la CPU.



Ejemplo para usar un mouse

Periférico: Un mouse conectado al sistema a través de un puerto USB.

Dispositivo: El mouse es el hardware que detecta el movimiento y los clics del usuario. Es un dispositivo de entrada porque envía datos al sistema.

Módulo de E/S: El controlador del mouse es el software que traduce las señales del mouse (como el movimiento o clics) en datos que el sistema operativo puede entender, como la posición del cursor y la acción de los clics.

CPU: La CPU utiliza los datos proporcionados por el controlador del mouse para mover el cursor en la pantalla y ejecutar acciones según los clics.

Gestión de Dispositivos y Controladores E/S

En Linux, casi todos los dispositivos de entrada y salida (E/S) se representan como archivos especiales ubicados en el directorio `/dev`. Estos dispositivos se clasifican en tres categorías principales: *dispositivos de carácter*, *dispositivos de bloque* y *dispositivos de red*.

Además, el sistema operativo Linux emplea un sistema de numeración conocido como *números mayor y menor* para identificar los dispositivos de forma eficiente y garantizar que las operaciones de lectura y escritura se realicen en el dispositivo correcto. En este texto, explicaremos estos conceptos y proporcionaremos ejemplos prácticos de cada tipo de dispositivo, explicando por qué se clasifican de esa manera.

Clasificación de Dispositivos E/S

1. Según Tipo de Acceso

Dispositivos de Carácter

Los dispositivos de carácter manejan flujos de datos secuenciales, procesando los datos byte a byte o carácter a carácter. A diferencia de los dispositivos de bloque, los dispositivos de carácter no permiten el acceso directo a bloques específicos de datos, ya que los datos se leen o escriben de manera continua en el orden en que son recibidos. Este tipo de dispositivos es adecuado para operaciones que requieren una transferencia continua de datos en tiempo real, como las interacciones con dispositivos de entrada y salida (E/S). Características:

- Los datos se procesan en el mismo orden en que son recibidos, sin la posibilidad de realizar acceso aleatorio.
- Los dispositivos de carácter no utilizan un búfer para almacenar temporalmente los datos, lo que implica que el sistema debe procesar los datos en tiempo real.
- La comunicación entre el dispositivo y el sistema operativo se realiza sin intermediarios, lo que reduce la latencia pero también limita la capacidad de optimización mediante caché.

Ejemplos comunes: Teclado, Ratón, Puertos serie (RS-232), Impresoras

Listar dispositivos de carácter:

```
ls -l /dev | grep '^c'
```

Dispositivos de Bloque

Los dispositivos de bloque almacenan y procesan datos en bloques de tamaño fijo (como 512 bytes o 4 KB). A diferencia de los dispositivos de carácter, los dispositivos de bloque permiten el acceso aleatorio a los datos, lo que significa que se puede leer o escribir directamente en cualquier bloque sin necesidad de procesar toda la secuencia de datos. Este acceso directo y la capacidad de almacenamiento en caché permiten una mayor eficiencia en la lectura y escritura de datos. Características:

- Los datos pueden leerse o escribirse directamente en cualquier bloque sin procesar los bloques anteriores.
- El sistema operativo utiliza un búfer para almacenar temporalmente los datos, lo que mejora el rendimiento de las operaciones de lectura y escritura. La información se organiza y gestiona en bloques de tamaño fijo, lo que facilita la recuperación y manipulación eficiente de datos.

Ejemplos comunes: Discos duros (HDD), Unidades de estado sólido (SSD), Memorias USB, CD/DVD

Listar dispositivos de bloque:

```
ls -l /dev | grep '^b'
```

Diferencias clave entre dispositivos de carácter y dispositivos de bloque

Característica	Dispositivos de Carácter	Dispositivos de Bloque
Modo de acceso	Secuencial	Aleatorio
Tamaño de datos	Procesado byte a byte	Procesado en bloques de tamaño fijo
Uso de caché	No utilizan caché	Utilizan caché para mejorar el rendimiento
Ejemplos	Teclado, mouse, puertos serie	HDD, SSD, USB, CD/DVD
Eficiencia	Adecuado para flujos de datos en tiempo real	Adecuado para grandes volúmenes de datos y acceso directo

Dispositivos de Red

Los dispositivos de red permiten la transmisión de datos entre dispositivos mediante protocolos de red. A diferencia de los dispositivos de carácter y bloque, los dispositivos de red trabajan con paquetes de datos en lugar de bytes o bloques. Características:

- Transmisión y recepción de datos mediante paquetes.
- Comunicación a través de protocolos de red (TCP/IP, UDP, etc.).
- Gestión mediante colas de entrada y salida.

Ejemplos comunes: Tarjetas de red Ethernet, Adaptadores Wi-Fi, Interfaz Loopback (lo)

Mostrar dispositivos de red activos:

`ip link show`

Significado de la primera letra en la salida de `ls -l`

Para ver todos los dispositivos y su tipo (bloque, carácter, directorio, etc.) en el directorio `/dev`, incluidos los ocultos (que comienzan con `.`) debes usar:

```
ls -l /dev
```

Cuando usas `ls -l`, el primer carácter de la primera columna indica el tipo de archivo o dispositivo:

Salida de ejemplo:

```
crw-rw-rw- 1 root root 1, 3 mar 13 10:35 null
brw-rw---- 1 root disk 8, 0 mar 13 10:35 sda
drwxr-xr-x 2 root root 4096 mar 13 10:35 pts
lrwxrwxrwx 1 root root 10 mar 13 10:35 cdrom -> /dev/sr0
prw-r--r-- 1 root root 0 mar 13 10:35 fifo
srw-rw-rw- 1 root root 0 mar 13 10:35 X11
```

Explicación:

- `c` → Dispositivo de carácter (`/dev/null`)
- `b` → Dispositivo de bloque (`/dev/sda` – disco duro)
- `d` → Directorio (`/dev/pts`)
- `l` → Enlace simbólico (`/dev/cdrom`)
- `p` → Pipe (`/dev/fifo`)
- `s` → Socket (`/dev/X11`)

2. Según su Función

- Dispositivos de Entrada

Los dispositivos de entrada permiten que los usuarios o sistemas externos envíen datos a una computadora. Estos dispositivos transforman la información del mundo físico a un formato que puede ser procesado por el sistema.

Ejemplos: Teclado, Ratón, Micrófono, Escáner, Cámaras de vigilancia, Sensores de IoT

- Dispositivos de Salida

Los dispositivos de salida permiten a las computadoras presentar la información procesada en una forma que sea comprensible para los usuarios. Su función principal es mostrar o reproducir los datos generados o procesados por el sistema.

Ejemplos: Monitor, Impresora, Altavoces, Pantallas táctiles, Actuadores de IoT

- Dispositivos de Entrada/Salida (E/S)

Estos dispositivos combinan tanto funciones de entrada como de salida, facilitando la comunicación bidireccional entre el sistema y los dispositivos externos. Son comunes en sistemas que requieren interacción continua y en tiempo real.

Ejemplos: Discos Duros, Pendrives/USBs, Redes IoT (Internet of Things), Tarjetas de red (Ethernet, Wi-Fi)

Números Mayor y Menor en Linux

En Linux, los dispositivos de hardware (discos duros, SSD, tarjetas de red, impresoras, terminales, etc.) no se manipulan directamente como en sistemas embebidos simples. En su lugar, cada dispositivo se representa como un archivo especial dentro del directorio `/dev/`.

Estos archivos permiten que los procesos interactúen con el hardware usando operaciones de E/S como `read()`, `write()` y `ioctl()`. Para identificar qué driver y qué instancia de dispositivo manejar, cada archivo de dispositivo tiene asignados dos números únicos: el número mayor (major number) y el número menor (minor number).

Estos números son gestionados internamente por el kernel, que asegura la correcta comunicación entre procesos y hardware, aislando a las aplicaciones del acceso directo a los dispositivos físicos y garantizando la seguridad y la estabilidad del sistema.

Número Mayor (Major Number)

El número mayor identifica el tipo o clase del dispositivo. Agrupa dispositivos similares bajo un mismo identificador, lo que permite al kernel saber qué controlador debe gestionar dicho dispositivo.

El major number no identifica al dispositivo individual, solo el tipo de dispositivo y el driver correspondiente.

Permite que el kernel agrupe dispositivos similares bajo un mismo manejador, simplificando la administración de recursos como interrupciones, buffers y mapeo de memoria.

Por ejemplo: Todos los discos duros pueden compartir el mismo número mayor, ya que son del mismo tipo de dispositivo.

Número Menor (Minor Number)

El número menor se utiliza para identificar dispositivos individuales dentro de una misma clase. Si tienes varios dispositivos del mismo tipo, cada uno tendrá un número menor diferente para diferenciarlo. Por ejemplo: Dos discos duros pueden compartir el mismo número mayor (por ser discos), pero tendrán números menores distintos para identificarlos individualmente.

```
amirnm@linuxadminhacks:~$ ls -l /dev/nvme0n1
brw-rw---- 1 root disk 259, 0 Jul 1 08:13 /dev/nvme0n1
amirnm@linuxadminhacks:~$ ls -l /dev/nvme0n1p1
brw-rw---- 1 root disk 259, 1 Jul 1 08:13 /dev/nvme0n1p1
amirnm@linuxadminhacks:~$ ls -l /dev/nvme0n1p2
brw-rw---- 1 root disk 259, 2 Jul 1 08:13 /dev/nvme0n1p2
amirnm@linuxadminhacks:~$
```

El gráfico muestra los **números mayores y menores**, que son clave en los drivers de Linux para identificar dispositivos.

- **Número Mayor (259):** Identifica el *tipo de dispositivo* y el *driver* que lo controla. En este caso, el **259** se asocia con el driver NVMe, que gestiona un tipo específico de disco de almacenamiento.
- **Número Menor (0, 1, 2):** Identifica una *instancia particular* de ese tipo de dispositivo. Por ejemplo, el **0** representa todo el disco, mientras que el **1** y el **2** representan sus particiones.

Para obtener información sobre los números mayor y menor de un dispositivo, se pueden utilizar los siguientes comandos:

ls -la /dev: Lista detallada de los dispositivos en /dev/ junto con sus números mayor y menor.
stat /dev/tty1: Muestra información detallada de un dispositivo específico.
/proc/devices: Muestra los drivers cargados y sus números mayores asociados

¿Para Qué Sirven los Números Mayor y Menor?

Los números mayor y menor permiten al sistema operativo gestionar y diferenciar dispositivos de manera eficiente:

- **Identificación:** Facilitan al kernel la distinción entre dispositivos.
- **Gestión de Recursos:** Ayudan al kernel a asignar recursos como memoria e interrupciones.
- **Acceso a Dispositivos:** Permiten a los programas interactuar con los dispositivos.

Ejercicios Resueltos sobre Clasificación de Dispositivos de Entrada y Uso de Llamadas al Sistema

Ejercicio 1: Leer un caracter del teclado

Entrada: El teclado es un dispositivo de entrada que permite al usuario ingresar datos.

Character: El teclado se considera un dispositivo de carácter, ya que se accede a los datos de uno en uno (un byte o un carácter a la vez)

System calls:

- **read():** La llamada al sistema read() se utiliza para leer un caracter del teclado.

Ejercicio 2: Leer la primera línea de un archivo de texto

Entrada: Leer un archivo es una operación de entrada

Character: Como esta leyendo la primera línea (carácter por carácter), se considera como dispositivo de carácter

System calls:

- **open():** La llamada al sistema open() se utiliza para abrir el archivo.
- **read():** La llamada al sistema read() se utiliza para leer datos del archivo
- **close():** La llamada al sistema close() se utiliza para cerrar el archivo.

Ejercicio 3: Leer la línea 80 de un archivo de texto de 100 líneas de longitud fija

Bloque: Es un dispositivo de bloque porque accedes a él en términos de bloques de datos

Entrada: El archivo se utilizaría como un dispositivo de entrada para leer su contenido

System calls:

- **open()**: se utiliza para abrir el archivo.
- **lseek()**: Se puede usar lseek() para ir a la posición correspondiente (en bytes) de la línea 80 y leerla de manera eficiente.
- **read()**: se utiliza para leer los datos desde la posición donde lseek() ha colocado el puntero.
- **close()**: se utiliza para cerrar el archivo.